Lab: Draft App Submission for App Store and Play Store

Estimated time: **15 minutes**

Welcome to this hands-on lab, where you will learn how to prepare and draft your Flutter app for submission to both the **Google Play Store** and **Apple App Store**. You will go through essential steps such as setting up your app's metadata, creating app bundles, testing the app, and understanding app store guidelines. Note that adding your application to Google Play Store and Apple App Store requires developer accounts that are not free. The instructions are shared in this lab and are optional to complete.

Objectives

After completing this lab, you will be able to:

- Prepare your Flutter app for submission to both the Google Play Store and Apple App Store
- Compare APKs and Android App Bundles
- Configure your app's metadata, pricing, and distribution settings
- Use the Google Play Console and App Store Connect for submission
- Conduct pre-launch testing using the Play Console and TestFlight

About Cloud IDE

Running the lab

This lab is designed to be completed using a Cloud IDE environment. You will be writing and executing Dart code directly within the IDE.

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands-on labs for course and project-related labs. It is an open-source IDE (Integrated Development Environment).

Important Notice about this lab environment

Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session to avoid losing your data.

Key terms

- **App Store Connect**: The platform used to manage and submit iOS apps to the Apple App Store.
- **Google Play Console**: The platform used to manage and submit Android apps to the Google Play Store.
- **Android App Bundle (AAB)**: The preferred packaging format for Android app distribution.
- **TestFlight**: A tool for beta testing iOS apps before release.
- **Pre-launch Testing**: A process to ensure the app works correctly across different devices and configurations.

Prerequisites

Before starting this lab, ensure you have the following setup in your Cloud IDE:

1. Create a new **Flutter** project:

   ```
   flutter create flutter_submission_lab
   cd flutter_submission_lab
   ```

2. Open the `lib/main.dart` file in your project.

Step 1: Prepare your app for Google Play Store submission

1. Generate a keystore file
   - Open the terminal.

- Run:

```
keytool -genkey -v -keystore release-key.jks -keyalg RSA -keysize 2048 -validity 10000 -alias key
```

- This command generates a secure file that stores your keys. You will be asked some identifying questions in the process. Keep note of the password you use here as you will need it later. Here is an example:

```
Enter keystore password:
 Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
 What is your first and last name?
   [Unknown]:  first last
 What is the name of your organizational unit?
   [Unknown]:  OU
 What is the name of your organization?
   [Unknown]:  Tech
 What is the name of your City or Locality?
   [Unknown]:  Montreal
 What is the name of your State or Province?
   [Unknown]:  Quebec
 What is the two-letter country code for this unit?
   [Unknown]:  CA
 Is CN=first last, OU=OU, O=Tech, L=Montreal, ST=Quebec, C=CA correct?
   [no]:  yes
 Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 10,000 days
         for: CN=first last, OU=OU, O=Tech, L=Montreal, ST=Quebec, C=CA
 [Storing release-key.jks]
```

2. Open the `android/app/build.gradle` file and configure signing for release. You will need to enter the keypassword from the previous step

```
android {
    ...
    defaultConfig { ... }
    signingConfigs {
        release {
            keyAlias 'my-key-alias'
            keyPassword 'your-key-password'
            storeFile file('your-keystore-path/my-release-key.jks')
            storePassword 'your-keystore-password'
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release
        }
    }
}
```

- keyPassword: you specified this password when prompted for Enter keystore password: during the keytool process.
- storePassword: this is the password for the keystore itself. It is typically the same password as the key password (unless you specify a separate password when creating the key alias).

3. The next step is to build the **Android App Bundle**. The Cloud IDE and the lab environment does not have the Android tooling installed to successfully build the apk bundle. The commands are available here if you want to follow on a machine that has Android installed.

```
flutter build appbundle --release
```

- This command creates an Android App Bundle (AAB), which is the preferred format for publishing on the Google Play Store.

4. Set up your app in the **Google Play Console**:
    - Go to the Google Play Console and create a new app.
    - Fill in all required information, including app name, default language, and contact details.
    - Configure the app's pricing and distribution settings.
    - Prepare your app's content rating by completing the questionnaire.
5. Upload the **Android App Bundle**:
    - Go to "App Bundle & APKs" in the Google Play Console.
    - Upload the `app-release.aab` file generated in the previous step.
6. Conduct pre-launch testing:
    - Enable the pre-launch report in the Play Console to test your app on various devices and configurations.
    - Review the test results and fix any reported issues before proceeding.

Step 2: Prepare your app for App Store submission

1. Set up your app in **App Store Connect**:
    - Go to App Store Connect and sign in with your Apple Developer account.
    - Click on "My Apps" and create a new app.
    - Fill in the required details, such as app name, primary language, and SKU.
    - Make sure the bundle ID matches the one used in Xcode.
2. Prepare your app for submission:
    - Open Xcode and archive your app by going to `Product > Archive`.
    - Once the archive is complete, click on "Distribute App" and select "App Store Connect".
    - Follow the prompts to upload the app binary to App Store Connect.
3. Use **TestFlight** for beta testing:
    - Go to the TestFlight section in App Store Connect.
    - Add a new build for internal or external testing.
    - Invite testers by entering their email addresses.
    - Review the feedback and make necessary improvements.
4. Submit your app for review:

- Navigate to "App Store" in App Store Connect.
- Select the build you want to submit and fill out the app's metadata, including screenshots and descriptions.
- Click on "Submit for Review" to send your app to Apple for approval.

Step 3: Monitor app performance and manage updates

1. Use the **Play Console** for post-release management:

   - Track your app's performance using metrics such as downloads, active users, crash reports, and user ratings.
   - Respond to user reviews promptly and address any issues.

2. Use **App Store Connect** analytics:

   - Monitor your app's performance, including installs, sessions, and user engagement.
   - Use Apple's analytics tools to track how users are interacting with your app.

3. Plan regular updates:

   - Regularly update your app to fix bugs, improve performance, and introduce new features.
   - Use CI/CD tools such as Bitrise or GitHub Actions to automate the deployment process.

Conclusion

Congratulations on completing this lab! You have learned how to prepare your Flutter app for submission to both the Google Play Store and Apple App Store. This includes setting up your app's metadata, managing app bundles, conducting pre-launch testing, and using app store management tools.